

Complexité algorithmique

Condensé de cours

1. Complexité et calculs usuels

1.1 Temps et espace

L'analyse de la complexité d'un algorithme vise à étudier la quantité de ressources nécessaires à son exécution.

Definition 1.1.1 (Complexité temporelle)

La **complexité temporelle** mesure le temps d'exécution, elle associe le nombre d'opérations effectuées à la quantité de données d'entrée n .

Definition 1.1.2 (Complexité spatiale)

La **complexité spatiale** mesure l'espace utilisé, elle associe l'espace utilisé à la quantité de données d'entrée n .

! But du cours

Notre but est d'analyser le comportement asymptotique de cette quantité en fonction de n . Pour connaître si notre algorithme s'exécute convenablement selon les contraintes des données d'entrée.

1.2 Notion de complexité

Soient C et C' appartenant à $\mathcal{A}(\mathbb{N}, \mathbb{N}^*)$:

Definition 1.2.1 (Grand O)

On dit que $C = \mathcal{O}(C')$, si et seulement si il existe un rang N et $\alpha \in \mathbb{R}^*$ tel que :

$$\forall n \geq N, 0 \leq C(n) \leq \alpha C'(n)$$

Lorsqu'on utilise la notation $\mathcal{O}$, on décrit une majoration de la complexité, soit le **pire des cas**.

Definition 1.2.2 (Theta)

On dit que $C = \Theta(C')$, si et seulement si il existe un rang N et $\alpha, \beta \in \mathbb{R}^*$ tel que :

$$\forall n \geq N, \alpha C'(n) \leq C(n) \leq \beta C'(n)$$

Lorsqu'on utilise la notation Θ , on décrit une **complexité moyenne**.

D'autres notations existent, mais ne seront pas réellement nécessaire par la suite ...

1.3 Calculs usuels

Soient $f, g, h \in \mathcal{A}(\mathbb{N}, \mathbb{N}^*)$, et $\alpha \in \mathbb{R}_+^*$:

Theorem 1.3.1

Nous avons les résultats suivants :

$$\mathcal{O}(f) + \mathcal{O}(g) = \mathcal{O}(f + g)$$

$$\mathcal{O}(f) \times \mathcal{O}(g) = \mathcal{O}(f \times g)$$

$$\mathcal{O}(f + \alpha) = \mathcal{O}(\alpha f) = \mathcal{O}(f)$$

$$\begin{cases} f = \mathcal{O}(g) \\ g \underset{+\infty}{\sim} h \end{cases} \implies f = \mathcal{O}(h)$$

$$\begin{cases} f = \mathcal{O}(g + h) \\ g = \underset{+\infty}{o}(h) \end{cases} \implies f = \mathcal{O}(h)$$

 **Attention aux constantes !**

Même si la complexité est optimale, une mauvaise constante peut vous empêcher de passer les tests d'un problème !

1.4 Classification des complexités

Nom	Complexité	Exemples d'utilisation
constante	$\mathcal{O}(1)$	Calcul d'aire d'un rectangle
logarithmique	$\mathcal{O}(\log(n))$	Recherche dichotomique dans une liste triée
linéaire	$\mathcal{O}(n)$	Recherche d'un éléments dans une liste
	$\mathcal{O}(n \log(n))$	Tri fusion ou avec tas
quadratique	$\mathcal{O}(n^2)$	Tri par insertion ou par sélection
polynomiale	$\mathcal{O}(n^k)$	Recherche d'un élément dans un tableau à k dimensions
exponentielle	$\mathcal{O}(2^n)$	N-ième terme de la suite de Fibonacci récursivement sans DP
factorielle	$\mathcal{O}(n!)$	Problème du voyageur de commerce implémenté naïvement

1.4 Classification des complexités

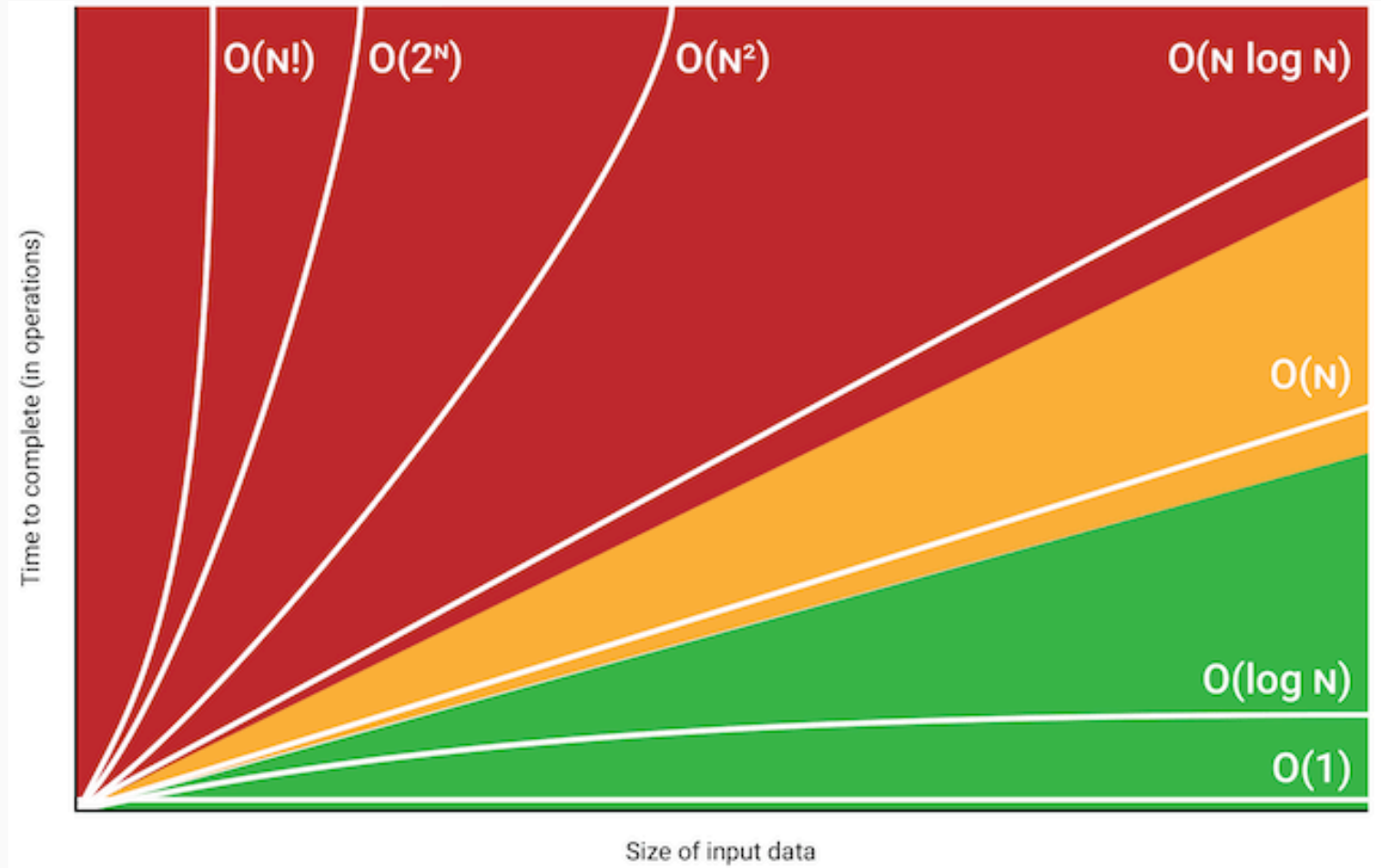


Figure 1: Comparaison des complexités de référence

2. Utilisation en pratique

2.1 Calcul de la complexité temporelle

Ce qui nous intéresse, c'est le fonctionnement de notre solution dans le pire des cas.

2.1 Calcul de la complexité temporelle

Ce qui nous intéresse, c'est le fonctionnement de notre solution dans le pire des cas.

Pour le calcul, il suffit de compter le nombre d'opérations effectuées selon la quantité de données.

En particulier on compte :

- Les **opérations arithmétiques** et **logiques** (*Opérations en $\mathcal{O}(1)$*).
- Les **conditions** avec le temps d'exécution de leur branche.
- Les **appels de fonction**, avec leur complexité.
- Les **boucles**, qui correspond à N itérations $\times M$ opérations.

2.1 Calcul de la complexité temporelle

Example.

```
1  def is_prime(n):  
2  
3      if n < 2:  
4          return False  
5  
6      sqrtn = int(n**.5)  
7  
8      for k in range(2, sqrtn + 1):  
9          if n % k == 0:  
10             return False  
11  
12         return True  
13
```

2.1 Calcul de la complexité temporelle

Example.

```
1  def is_prime(n):
2
3      if n < 2:
4          return False
5
6      sqrtn = int(n**.5)
7
8      for k in range(2, sqrtn + 1):
9          if n % k == 0:
10             return False
11
12         return True
13
```

La complexité dans le pire des cas de cet algorithme est $\mathcal{O}(\sqrt{n})$.

2.2 Pourquoi l'utiliser ?

Le calcul de la complexité de votre solution détermine précisément si elle a une chance de passer le jeu de tests.

2.2 Pourquoi l'utiliser ?

Le calcul de la complexité de votre solution détermine précisément si elle a une chance de passer le jeu de tests.

💡 Quelle complexité choisir ?

Vous pouvez deviner la complexité à avoir en lisant les contraintes de temps, et celles sur les données d'entrées.

2.2 Pourquoi l'utiliser ?

Le calcul de la complexité de votre solution détermine précisément si elle a une chance de passer le jeu de tests.

💡 Quelle complexité choisir ?

Vous pouvez deviner la complexité à avoir en lisant les contraintes de temps, et celles sur les données d'entrées.

① Valeur importante

Les machines de tests sont généralement configurées pour faire 10^8 opérations par seconde en C++.

2.3 Quelques données

Quantité de données (n)	Complexité temporelle possible
$n \leq 10$	$\mathcal{O}(n!)$, $\mathcal{O}(n^6)$, $\mathcal{O}(n^7)$
$n \leq 20$	$\mathcal{O}(2^n \cdot n)$, $\mathcal{O}(n^5)$
$n \leq 80$	$\mathcal{O}(n^4)$
$n \leq 500$	$\mathcal{O}(n^3)$
$n \leq 7500$	$\mathcal{O}(n^2)$
$n \leq 7 \cdot 10^4$	$\mathcal{O}(n\sqrt{n})$
$n \leq 5 \cdot 10^5$	$\mathcal{O}(n \log(n))$
$n \leq 5 \cdot 10^6$	$\mathcal{O}(n)$
$n \leq 10^{18}$	$\mathcal{O}(\log(n))$, $\mathcal{O}(\log^2(n))$, $\mathcal{O}(1)$

3. Crédits

Auteur et images : Bilel MAHDJOUR
Pour le **Bureau des technologies** de
Polytech Nice Sophia

4. Sources

- USACO guide - time complexity
- CP algorithms